Chapter 1

Introduction to OOP and C#

ENEL712 Embedded Systems Design — Lecture 14

Summary

Foundational OOP concepts (classes/objects, encapsulation, get/set, constructors, overloading, reference types) and an introduction to C# and the .NET platform.

Topics Covered

- OOP overview
- Classes, objects, properties, methods
- Encapsulation and access control
- C# memory and initialization mechanics
- Introduction to C#
- Capabilities and use cases
- Why use C# (and why not)

1.1 OOP Overview

- OOP organises programs around objects and data rather than actions and logic.
- Modularity, reusability, encapsulation, and hierarchy.
- Other OOP languages: C++, Java, MATLAB, Python, PHP.

1.2 Classes, Objects, Properties, Methods

- Class: template/blueprint integrating both code and data.
- Object: specific instance of a class — created with the new keyword.
- Property (data member): a data field, e.g., array.Length.
- Method: a function the object can perform, e.g., account.Open().

1.3 Encapsulation and Access Control

- Encapsulation binds methods and properties together and controls external access.
- Modifiers: private (default) hides data; public exposes properties/methods.
- Get/Set methods provide controlled access — allowing validation, conversion, or other processing.

1.4 C# Memory and Initialization Mechanics

- Classes are reference types. Declaring a variable creates a reference on the stack; new allocates the object on the heap.
- Constructor: special method run on instantiation — same name as the class, no return type.
- Overloading: multiple methods/constructors with the same name and different parameter signatures.

1.5 Introduction to C#

- Modern, general-purpose, simple OOP language built by Microsoft (2001), evolved from FORTRAN/C/C++.
- Built on the .NET framework — originally Windows-only, increasingly Linux/Mac via cross-platform .NET.
- C# is compiled to an intermediate language and then to machine code at runtime.

1.6 Capabilities and Use Cases

- GUI development for Windows.
- Web/cloud (ASP.NET, Azure) and Silverlight programs.
- Office add-ins, libraries, and integration with C/C++/Java/FORTRAN/MATLAB.
- Case study: JSteam — integrates C++/C#/Excel with custom ribbons, functions, and graphical plant models (HP/MP/LP users).

1.7 Why Use C# (and Why Not)

1.7.1 Advantages

- CLI-based — fast app development with rich libraries.
- Simpler than C++ with great Office/Cloud integration.
- Managed code: automatic GC and platform independence.

1.7.2 Disadvantages

- Not ideal for high-speed numerical work, compression, or encoding (no direct compile to native code).
- Historically Windows-only; improved by open-source .NET (2014) and cross-platform Visual Studio (2015).

1.8 Use Cases & Applications

- Modelling real-world entities (Account, Sensor, Order) as classes keeps related data and behaviour together — easier to reason about than passing big structs through dozens of free functions.
- Reusability across products: a Sensor base class with TemperatureSensor, HumiditySensor children means the rest of the application code doesn't need to know which model it's reading from.
- C# has first-class GUI, networking, and database libraries, making it strong for desktop tools and the PC side of an embedded product.
- Cross-platform with modern .NET: the same C# can run on Windows, Linux, and macOS — useful for companion apps that ship to multiple OSs.

1.9 Worked Example: A Tiny Account Class

```
public class Account
{
    private decimal balance;
    private string owner;

    public Account(string owner, decimal opening)
    {
        this.owner   = owner;
        this.balance = opening;
    }

    public void Deposit(decimal amount)
    {
        if (amount <= 0) throw new ArgumentException("positive only");
        balance += amount;
    }

    public bool Withdraw(decimal amount)
    {
        if (amount <= 0 || amount > balance) return false;
        balance -= amount;
        return true;
    }

    public decimal Balance => balance;
}
```

The fields are private (no caller can corrupt the balance), the constructor enforces a valid initial state, and Withdraw/Deposit each maintain the invariant that balance never goes negative.

1.10 C# vs C++ vs C — Quick Compare

Trait	C	C++	C#
Memory	Manual	Manual / RAII	Garbage-collected
Compiled to	Native	Native	CIL → JIT to native
GUI ease	Hard	Hard (Qt etc.)	Easy (WinForms, WPF, MAUI)
Performance	Highest	Very high	Slightly lower (managed runtime)
Best for	Embedded firmware	Performance-critical apps with OOP	Desktop apps, services, tooling

1.11 Intuition — Why OOP Feels Strange at First

Coming from structured/procedural programming, OOP often feels disordered — 'a ball of clay thrown at the wall' rather than the linear flow you are used to. The reason it works is that programs are organised around data lifetimes (stack vs heap) and encapsulation, not around the order of statements. Once you map every class decision back to 'where does this live in memory and who is allowed to touch it?', the apparent chaos becomes deliberate structure.

- Structured programming organises code by control flow.
- OOP organises code by data ownership and lifetime.
- Both compile down to the same machine instructions; the difference is at design time.

1.12 Value Types vs Reference Types — In Practice

Value-type assignment copies the bits. Reference-type assignment copies the pointer — both names now refer to the same heap object. Mutating one is visible to the other.

```
// Value type -- independent copies
int c = 100;
int d = c;          // d gets its own 100
d = 200;            // c is still 100

// Reference type -- shared object
int[] a = { 1, 2, 3 };
int[] e = a;        // e and a point at the SAME array
e[0] = 99;          // a[0] is now 99 too

// Decoupling a reference: make a real copy
int[] f = new int[a.Length];
Array.Copy(a, f, a.Length);   // f is independent of a
```

- Strings, arrays, and class instances are reference types — assignment shares them.
- int, double, bool, char, struct, enum are value types — assignment copies them.
- Array.Copy or new T[] { ... } when you need a real copy.
- Array.Sort sorts in place — every reference observes the sorted result.

1.13 Mutable vs Immutable Types

- Mutable: contents can change after construction (List<T>, arrays, most classes).
- Immutable: contents are fixed once created (string, ValueTuple, record with init-only members).
- Mutable + shared reference is the classic source of 'spooky action at a distance' bugs.
- Immutable types are inherently thread-safe — no synchronisation needed.
- Default to immutable when the data is small and conceptually a value (point, money amount, configuration).

1.14 Picking a Type — var, decimal, double, int

- int: 32-bit whole numbers — the default for counters, indices, sizes.
- double: 64-bit floating-point — for measurements, scientific calculations, anywhere precision tolerates rounding error.
- decimal: 128-bit fixed-point — money and other values where rounding errors would be unacceptable.
- Don't allocate a double when you only need an int — you are using 8 bytes for what fits in 4.
- var asks the compiler to infer the type from the initialiser — useful for long generic types and anonymous objects, never for code reviewers' convenience.

```
var person = new { Name = "Bob", Age = 25 };   // anonymous type -- must use var
var list = new List<KeyValuePair<int, string>>(); // long type, var is cleaner
int count = 0;                                    // simple -- be explicit
```

1.15 Methods vs Properties — Reading the Brackets

In Visual Studio's IntelliSense list, the trailing brackets tell you what something is. Brackets ⇒ method (an action). No brackets ⇒ property/field (a value).

Member	Brackets?	Meaning
array.Length	no	Property — read the current length
array.Sort()	yes — empty	Method — performs an action with no inputs
account.Deposit(50)	yes — with arg	Method — action that takes input
account.Balance	no	Property — exposes the balance

1.16 Abstraction — The House / Street / Neighbourhood Analogy

Abstraction means choosing the right level of detail for the question being asked.

- Standing in front of the door: the house number matters.
- Driving in the suburb: the street name matters; the house number is noise.
- Crossing the city: the neighbourhood matters; street and number are noise.

- .NET works the same way — your code talks about Console.WriteLine; the runtime worries about which registers, pages, and I/O ports it touches.
- Good APIs hide the level below; you only drop a level when the level you are at can no longer answer your question.

1.17 Namespaces — The Same-Realm Rule

- Everything inside the same namespace can refer to each other by simple name.
- Code in a different namespace must either fully qualify the name or use a using directive.
- Same role as #include for organising C, but stricter — the namespace is part of the type's identity, not just a copy/paste of declarations.
- Multiple files can share one namespace (e.g., one System.IO across hundreds of source files).
- A namespace is not the same as an assembly: a NuGet package is one assembly that may expose several namespaces.

1.18 Encapsulation — The Rainbow's End Analogy

A theme park makes encapsulation concrete: the park encloses the rides (grouping), and entry is only via the ticket office (controlled access).

- The park boundary = the class itself.
- The ticket office = a public method or property — the only sanctioned entry point.
- Climbing the fence = bypassing encapsulation (reflection, public fields) — possible but discouraged.
- Not every part is private: the parking lot is public — everyone can use it (analogous to truly public helper methods).
- If the park ceases to exist, so do its rides — when a class instance is collected, its private state goes with it.

Key Definitions

OOP
Programming model organised around objects and data.

Object
Instance of a class created with the new keyword.

Class
Template/blueprint integrating code and data.

Encapsulation
Bundling methods/properties and controlling access — info hiding.

Modularity
Structuring code into organized, independent components.

Method
Function an object can perform.

Property
Data field/member that holds an object's data.

Constructor
Method that runs automatically when an object is created.

Managed Code
Code that runs under the CLR with automatic garbage collection.

CLI
Common Language Infrastructure — standardized infrastructure C# implements.

Garbage-Collected Language
Language whose runtime automatically frees memory that no live variable references.

Managed Code
Code that runs under a runtime (the CLR for C#) which provides services like type safety and GC.

Reference Type
Class instance whose variable holds an address to the actual data on the heap.

Constructor Chaining
Calling another constructor of the same or base class (this(...) / base(...)) to share initialisation logic.

Value type
Type whose values are stored directly (int, double, struct, enum). Assignment copies the value.

Reference type
Type whose values are accessed through a reference (class, array, string, delegate). Assignment copies the reference, not the data.

Mutable type
Type whose contents can be changed after construction.

Immutable type
Type whose contents are fixed at construction; any 'change' produces a new instance.

Abstraction
Hiding lower-level detail behind an interface that exposes only what the caller needs at this level.

Namespace
Logical grouping that scopes type names to avoid collisions; required to share types across files.

Sources

- Lecture 14.pdf